

Лабораторна робота № 11 з курсу

"Архітектура обчислювальних систем та комп'ютерна схемотехніка"

Виконав:

Студент групи ПМі-12

Козій Дем'ян

Тема: Масиви з індексуванням, стек, робота з матрицями

Мета роботи: Використовуючи середовище Visual Studio, написати програму опрацювання масивів даних на мові Assembler

Варіант: 11

Хід виконання роботи:

1. У середовищі Visual Studio створено C++ проект для реалізації базового завдання з множення матриць
2. З використанням інструменту MASM у вигляді асемблерної вставки `__asm` реалізовано алгоритм множення двох квадратних матриць. Для доступу до елементів двовимірного масиву (масиву вказівників) використано складну базово-індексну адресацію (наприклад, `[eax + 4 * edx]`). Для збереження проміжних значень адрес і регістрів у вкладених циклах активно використовувався стек (команди `push` та `pop`)
3. Для виконання індивідуального завдання (варіант 11) було розроблено автономну 64-бітну програму на асемблері NASM
4. У програмі на NASM реалізовано динамічне виділення пам'яті для масиву вказівників та рядків матриці за допомогою викликів зовнішньої C-функції `malloc`. Введення розмірності та елементів матриці реалізовано через зовнішню функцію `scanf`
5. Розроблено алгоритм пошуку максимального елемента a_{kl} , який задовольняє першу умову: при діленні на 4 дає остачу 3. Для цього використано команду цілочисельного ділення зі знаком `idiv` (з попереднім розширенням знаку командою `cdq`)
6. Додано фінальний блок перевірки другої умови завдання: якщо елемент знайдено, програма обчислює суму $1 + k + 1$, виконує ділення максимального елемента на цю суму і перевіряє, чи дорівнює остача 3. Відповідний результат виводиться на екран за допомогою `printf`

Програмний код:

1. Базове завдання (C++ з MASM-вставкою):

```
#include <iostream>
#include <Windows.h>
using namespace std;

int main() {
    SetConsoleCP(CP_UTF8);
    SetConsoleOutputCP(CP_UTF8);

    int n;
    cout << "Введіть n: ";
    cin >> n;
    int** a = new int* [n];
    int** b = new int* [n];
    int** res = new int* [n];
    cout << "Введіть елементи матриці A: ";
    for (int i = 0; i < n; i++) {
        a[i] = new int[n];
        b[i] = new int[n];
        res[i] = new int[n];
        for (int j = 0; j < n; j++) {
            cin >> a[i][j];
            b[i][j] = i + j - 1;
            res[i][j] = 0;
        }
    }
    cout << "A : " << endl;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) { cout << a[i][j] << " "; }
        cout << endl;
    }

    cout << endl;
    cout << "B : " << endl;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cout << b[i][j] << " ";
        }
        cout << endl;
    }

    cout << "A * B: " << endl;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cout << res[i][j] << " ";
        }
        cout << endl;
    }

    return(0);
}
```

```
__asm {
    mov ecx, 0
    start1:
    mov edi, 0
start2:
    mov edx, res
    mov esi, ecx
    imul esi, 4
    add edx, esi
    push[edx]
    pop edx
    mov esi, edi
    imul esi, 4
    add edx, esi
    push edx
    xor esi, esi
start:
    mov eax, a
    mov ebx, b
a_to_eax:
    mov edx, ecx
    mov eax, [eax + 4 * edx]
    mov edx, esi
    mov eax, [eax + 4 * edx]
b_to_ebx:
    mov edx, esi
    mov ebx, [ebx + 4 * edx]
    mov edx, edi
    mov ebx, [ebx + 4 * edx]
end:
    pop edx
    imul eax, ebx
    add[edx], eax
    push edx
    inc esi
    cmp esi, n
    jne start
    pop edx
    inc edi
    cmp edi, n
    jne start2
    inc ecx
    cmp ecx, n
    jne start1
}
```

2. Індивідуальне завдання (Варіант 11, NASM 64-bit):

```
default rel

section .data
    fmt_in    db "%d", 0
    fmt_out   db "%d ", 0
    fmt_newline db 10, 0

    msg_fail  db "Умова не виконалась! Елемент %d на позиції [%d][%d] не дає остачу 3.", 10,
    msg_ok    db "Умова виконалась! Елемент %d підходить.", 10, 0
    msg_none  db "Елементів з остачею 3 при діленні на 4 не знайдено.", 10, 0

section .bss
    n resd 1
    array_pointer resq 1

    max_val resd 1
    max_i resq 1
    max_j resq 1
    found resb 1

section .text
    global main
    extern scanf
    extern printf
    extern malloc
    extern free
    extern SetConsoleOutputCP
```

```
main:
    sub rsp, 40

    mov rcx, 65001
    call SetConsoleOutputCP

    lea rcx, [fmt_in]
    lea rdx, [n]
    call scanf

    movsxd rax, dword [n]
    mov rbx, 8
    mul rbx
    mov rcx, rax
    call malloc
    mov [array_pointer], rax

    xor rbx, rbx
.alloc_rows:
    movsxd rcx, dword [n]
    cmp rbx, rcx
    jge .input_matrix

    mov rax, 4
    movsxd rcx, dword [n]
    mul rcx
    mov rcx, rax
    call malloc
```

```
    mov rcx, [array_pointer]
    mov [rcx + rbx*8], rax

    inc rbx
    jmp .alloc_rows

.input_matrix:
    xor r12, r12
    call scanf

    movsxd rax, dword [n]
    cmp r12, rax
    jge .print_matrix

    xor r13, r13
    .in_col:
    movsxd rax, dword [n]
    cmp r13, rax
    jge .in_row_next

    mov rcx, [array_pointer]
    mov rdx, [rcx + r12*8]
    lea rdx, [rdx + r13*4]

    lea rcx, [fmt_in]
    call scanf

    inc r13
    jmp .in_col
```

```
.in_row_next:
    inc r12
    jmp .in_row

.print_matrix:
    xor r12, r12
.out_row:
    movsxd rax, dword [n]
    cmp r12, rax
    jge .find_max

    xor r13, r13
.out_col:
    movsxd rax, dword [n]
    cmp r13, rax
    jge .out_row_next

    mov rcx, [array_pointer]
    mov rdx, [rcx + r12*8]
    mov edx, dword [rdx + r13*4]

    lea rcx, [fmt_out]
    call printf

    inc r13
    jmp .out_col
```

```
.out_row_next:
    lea rcx, [fmt_newline]
    call printf
    inc r12
    jmp .out_row

.find_max:
    mov byte [found], 0

    xor r12, r12
.search_row:
    movsxd rax, dword [n]
    cmp r12, rax
    jge .check_condition

    xor r13, r13
.search_col:
    movsxd rax, dword [n]
    cmp r13, rax
    jge .search_row_next

    mov rcx, [array_pointer]
    mov rdx, [rcx + r12*8]
    mov eax, dword [rdx + r13*4]
```

```
    mov ecx, eax
    cdq
    mov r8d, 4
    idiv r8d
    cmp edx, 3
    jne .search_col_next

    cmp byte [found], 0
    je .update_max

    cmp ecx, dword [max_val]
    jle .search_col_next

.update_max:
    mov byte [found], 1
    mov dword [max_val], ecx
    mov qword [max_i], r12
    mov qword [max_j], r13

.search_col_next:
    inc r13
    jmp .search_col

.search_row_next:
    inc r12
    jmp .search_row
```

```
.check_condition:
    cmp byte [found], 0
    je .not_found

    mov rax, 1
    add rax, [max_i]
    add rax, [max_j]
    mov r8d, eax

    mov eax, dword [max_val]
    cdq
    idiv r8d
    cmp edx, 3
    je .success

    lea rcx, [msg_fail]
    movsxd rdx, dword [max_val]
    mov r8, [max_i]
    mov r9, [max_j]
    call printf
    jmp .end
```

```
.success:
    lea rcx, [msg_ok]
    movsxd rdx, dword [max_val]
    call printf
    jmp .end

.not_found:
    lea rcx, [msg_none]
    call printf

.end:
    add rsp, 40
    ret
```

Результати виконання програми:

Для перевірки коректності роботи було проведено тестування обох програм.

Результат роботи базового завдання:

```
Введіть n: 2
Введіть елементи матриці A: 1 2 3 4
A :
1 2
3 4

B :
-1 0
0 1

A * B:
-1 2
-3 4
```

Результат роботи індивідуального завдання:

```
PS E:\Uni\OC\asm\class5> ./class5.exe
3
1
2
3
4
5
7
1
2
3
1 2 3
4 5 7
1 2 3
Умова виконалась! Елемент 7 підходить.
```

Висновок:

Під час виконання лабораторної роботи № 11 було засвоєно принципи роботи з масивами та матрицями на рівні мови асемблера. Опановано використання базово-індексної адресації для доступу до елементів двовимірних структур даних у пам'яті. Практично закріплено навички роботи зі стеком для тимчасового збереження даних, що дозволило уникнути конфліктів регістрів у вкладених циклах.

В індивідуальному завданні успішно реалізовано взаємодію з зовнішніми C-функціями (malloc, scanf, printf), динамічне виділення пам'яті під матрицю, використання команд знакового ділення (idiv) та обробку складних логічних умов з використанням індексів матриці. Це дозволило повністю виконати умови

поставленого завдання та зрозуміти низькорівневі механізми роботи багатовимірних масивів.